Atlas: Multi-Agent Research & Brief Generator

Manasi Mathkar · Wipro Junior FDE Pre-screening Assignment · May 2026

Use case: Atlas takes a research question (e.g., “What is the current state of solid-state battery commercialization?”) and returns a sourced one-page brief. Five specialized agents collaborate through a LangGraph state machine, with three security checkpoints and hard budget caps on autonomy.

1. Multi-Agent Architecture

Five agents are orchestrated as a directed graph with one fan-out (parallel research) and one bounded loop (critic to writer). The graph topology is explicit (the orchestrator is not itself an LLM), which keeps control flow auditable and budget-bounded.

Agent	Role	Tools	Model
Planner	Decompose user query into ≤5 sub-questions	none	Claude Sonnet
Researcher (×N)	Parallel web search + summarize per sub-Q	Tavily search (allowlisted)	Claude Sonnet
Writer	Synthesize sourced brief from findings	none	Claude Sonnet
Critic	Fact-check vs. sources; emit confidence score; loop if weak	none	Claude Sonnet
Security	Input / web-content / output checkpoints	regex + LLM judge	Claude Haiku

Communication: All agents read and write a single typed *GraphState* (TypedDict). The *findings* field uses a list-merge reducer so parallel Researcher writes don’t race. Agents never call each other directly; the graph router controls every transition.

Execution model: Sequential at the Planner / Writer / Critic boundaries; parallel fan-out for Researchers (one per sub-question, up to five). The Critic→Writer loop is capped at one retry. This mixes hierarchy (orchestrator decides routes) with peer collaboration (Critic can reject the Writer’s output) without giving any agent unilateral control.

2. Security, Safety, and Guardrails

The dominant threat for a research agent is not user prompt injection; it is **indirect prompt injection via fetched web pages**, because attackers can plant payloads in any page Tavily might return. Guardrails are layered, with three independently testable checkpoints:

- **Input guardrail:** regex pass for known injection markers, escalated to a Haiku-based classifier on ambiguous matches. Sensitive but legitimate research questions pass; injection/jailbreak attempts are blocked with a typed refusal.
- **Web-content guardrail:** every fetched source is HTML-stripped (bleach), then scanned for injection markers. Flagged sources are dropped from the synthesis bundle; the Writer is also instructed to treat all source content as quoted data, not instructions.
- **Output guardrail:** PII regex (SSN, credit card, email, phone, API-key shapes) auto-redacts before display; a Haiku-based policy classifier blocks instructions for serious harm or fabricated quotes from named real people.

Non-LLM defenses: (1) **Tool allowlisting**: only the Researcher module imports the Tavily client; no agent has shell, raw HTTP, or filesystem access. (2) **Hard budget caps** loaded from config: ≤5 sub-questions, ≤10 search calls, ≤1 critic retry, 4k tokens/agent, 120s request timeout. (3) **Structured audit logging** via structlog: every agent step, model call, search call, and guardrail decision is logged with stable keys for review. (4) **Container hardening**: the Docker image runs as a non-root user; secrets are injected as Lightsail environment variables, never committed (.env is git-ignored).

3. Implementation Approach

Stack: Python 3.11 · LangGraph (orchestrator) · Anthropic SDK (Claude Sonnet for reasoning, Haiku for cheap classifiers) · Tavily (web search) · FastAPI + Server-Sent Events with a custom HTML/JS frontend (live-streaming UI) · pydantic-settings (typed config) · tenacity (retries) · structlog (audit logs) · bleach (HTML sanitization).

Why LangGraph: the explicit graph makes the architecture diagram and the audit trail line up exactly, parallelism is a first-class concept (Send), and security checkpoints are nodes, not middleware bolted on. CrewAI and AutoGen are higher-level but obscure exactly the control-flow story this assignment asks for.

Lifecycle: Each run instantiates a compiled graph and invokes it with an initial state; agents are pure stateless functions so the orchestrator owns memory. Termination is guaranteed by structural means: critic_retries counter, search_calls budget, request timeouts, and a single END node.

Errors / retries: LLM calls go through a single wrapper with tenacity exponential backoff on transient errors (rate-limit, timeout, 5xx). JSON parsing failures degrade gracefully to a tagged error object so downstream nodes can route. Search-tool failures yield an empty SubFinding with a reason, so the run continues with whatever remains.

Testing: Pytest suite covers (i) the regex guardrail corpus (positives: known jailbreak patterns; negatives: legitimate sensitive-topic queries), (ii) PII regex / redaction, (iii) graph compiles and has the expected node set, (iv) the search tool flags injected content and respects the budget. Golden-set query runs are scripted but require live API keys.

Deployment: The Dockerfile builds an image running uvicorn/FastAPI on port 8080. The deploy target is AWS Lightsail Containers (Oregon): the image is pushed to a public registry, then a Lightsail container service runs it with the two API keys injected as environment variables, exposing a public HTTPS URL with a /healthz health check. The non-root user, dependency pinning in pyproject.toml, and .dockerignore prevent secret leakage.

4. Use of AI/LLMs and Collaboration

Where LLMs are used: decomposition (Planner), grounded summarization with citations (Researcher), synthesis (Writer), fact-checking (Critic), and classification (input and output guardrails). LLMs do *not* decide control flow, do not pick tools at runtime, and cannot escalate privileges; those are graph-router decisions made by deterministic Python.

Collaboration / negotiation: The Critic is an explicit adversarial peer to the Writer. If the Critic identifies more than one unsupported claim it loops back to the Writer with structured issues to address; if the second pass still has issues, the run proceeds but surfaces them to the user. The Critic also emits a 0 to 100 confidence score (weighing source grounding, source quantity and agreement, and issues found) that is shown alongside the brief, so the reader sees not just an answer but a calibrated trust signal. This negotiation is bounded; there is no free-form agent chat that could spiral.

Autonomy vs. control: Atlas deliberately sits at the low-autonomy end of the spectrum. Agents have narrow roles (one objective, one output schema), no tool choice (allowlist), and no memory across runs. The trade-off is less emergent behavior on truly novel queries, accepted because the use case rewards reliability, auditability, and safety over open-ended exploration. The same skeleton (typed state, allowlisted tools, layered guardrails, capped loops) is what scales to higher-stakes domains.

Repository: github.com/manasimathkar/atlas-multi-agent **Live demo:** wipro-manasi.jc3b1dk9p8244.us-west-2.cs.amazonlightsail.com